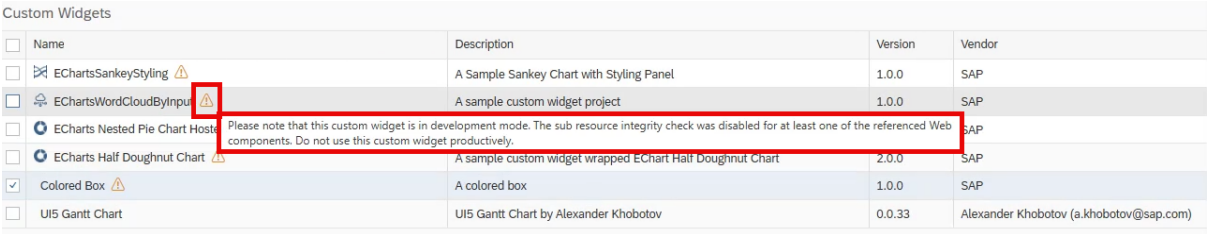
How to change SAC custom widgets from development- to production status

This example walks you though the exercise of turning the “UI5GanttChart” to a integer mode.

The custom widgets that are provided from SAP on GitHub are all set to development mode. This can be easily recognized by hovering over the attention symbol in the custom widget listing on SAC.



This is the result of the “ignoreIntegrity” setting in the “.json” definition of the widget. As default this is set to “true”.

```
"webcomponents": [
    {
        "kind": "main",
        "tag": "ui5-gantt-chart",
        "url": "/UI5GanttChart.js",
        "integrity": "",
        "ignoreIntegrity": true
    },
```

To set the custom widget into a production state the “ignoreIntegrity” flag must be set to “false” and the “integrity” needs to be filled with a hash code.

Be aware that this setting must be done for each JS component that is part of the custom widget. In my example there are 3 parts:

- UI5GanttChart.js
- UI5GanttChartStyling.js
- UI5GanttChartBuilder.js

The Hash code is like a certificate that ensures that the code of the “.js” component have not been changed or modified since the Hash was created.

This procedure will get you the Hash that is valid as long as the “.js” files are not changed.

- Download the custom widget ".json" and ".zip" file from GitHub and place it in directory of your choice.
- Unzip the compressed file to get the ".js" files.
- Create a subfolder in your directory and name it Build.
- Change into the Build directory
- Create a text file and name it "sign.js"
- Copy the following code into it.

```
const fs = require('fs')
const { createHash } = require('crypto')
const path = require('path')

const run = js => {
  const hashSha256 = createHash('sha256')
  const input = fs.createReadStream(path.join(__dirname, '../', js))
  input.on('readable', () => {
    const data = input.read()
    if (data) {
      hashSha256.update(data)
    } else {
      const integrity = `sha256-${hashSha256.digest('base64')}`
      console.log(` ${js}: ${integrity}` )
    }
  })
}

[
  'UI5GanttChart.js',
  'UI5GanttChartBuilder.js',
  'UI5GanttChartStyling.js'
].forEach(js => run(js))

// The result could look like this:
// UI5GanttChart.js: sha256-
v/nGZRJq2dtSKF0gmLOtJ3igE8b8n8UaEqvaUxrDUOI=
// UI5GanttChartStyling.js: sha256-
kOhpeMUQQwtF9QH9Lwv8bPP/VdpHo9UGCL02bdc0SmE=
// UI5GanttChartBuilder.js: sha256-
eWXDDWBMoMYRkBmT879liabb+x7dyZWovYomMG1o0M0=
```

- Make sure that all your ".js" components are listed in the marked section.

- To execute the script, you need to have nodeJS installed on your machine. If so open a terminal (cmd.exe on Win) change to the directory where your “sing.js” is located. Execute it with “node sign.js”. the result shout look like this:

  UI5GanttChart.js: sha256-v/nGZRJq2dtSKF0gmLOtJ3igE8b8n8UaEqvaUxrDUOI=
  UI5GanttChartStyling.js: sha256-kOhpeMUQQwtF9QH9Lwv8bPP/VdpHo9UGCL02bdc0SmE=
  UI5GanttChartBuilder.js: sha256-eWXDDWBMoMYRkBmT879liabb+x7dyZWovYomMG1o0M0=

- Copy the Hash, starting with “sha256”, so you can use it later.
- If you do not have nodeJS on your machine just copy the whole directory where you have placed your widget including of the “Build” subdirectory to a memory stick or a shared drive.

Build	09.04.2025 11:54	File folder	
UI5GanttChart.js	09.04.2025 12:13	Text Document	15 KB
UI5GanttChart.json	09.04.2025 13:42	JSON File	6 KB
UI5GanttChart.zip	09.04.2025 13:35	Compressed (zipp...	9 KB
UI5GanttChartBuilder.js	09.04.2025 12:13	Text Document	12 KB
UI5GanttChartStyling.js	09.04.2025 12:13	Text Document	5 KB

  Find a machine that has nodeJS installed, open the copied “Build” directory and execute the “sign.js”. Copy the result as above and go back to your system.
- Find your widget “.json” file and open it for editing.
- You need to change the “ignoreIntegrity” flag to “false” for each of your “.js” components.
  In this example these are the 3 listed on page one.
- For each “.js” component the related and copied Hash must be copied into the “integrity” flag. Save the file and exit.
  The section on our “UI5GanttChart.json” should now look like this:

```
"webcomponents": [
    {
        "kind": "main",
        "tag": "ui5-gantt-chart",
        "url": "/UI5GanttChart.js",
        "integrity": "sha256-v/nGZRJq2dtSKF0gmLOtJ3igE8b8n8UaEqvaUxrDUOI=",
        "ignoreIntegrity": false
    },
    {
        "kind": "styling",
        "tag": "ui5-gantt-chart-styling",
        "url": "/UI5GanttChartStyling.js",
        "integrity": "sha256-kOhpeMUQQwtF9QH9Lwv8bPP/VdpHo9UGCL02bdc0SmE=",
        "ignoreIntegrity": false
    },
    {
        "kind": "builder",
        "tag": "ui5-gantt-chart-builder",
        "url": "/UI5GanttChartBuilder.js",
        "integrity": "sha256-eWXDDWBMoMYRkBmT879liabb+x7dyZWovYomMG1o0M0=",
        "ignoreIntegrity": false
    }
],
```


- If you have changed something on the ".js" files before creating the Hash and after uncompressing it from the original, then you need to first compress all the ".js" files to a ZIP again.
- Now you are ready to upload the integer version of the custom widget to the SAC.

The result of your work should be reflected on the custom widget listing in the SAC. The warning sign should not come up for your integer widget.